

UNIVERSITATEA TEHNICĂ DE CONSTRUCȚII BUCUREȘTI

Facultatea de Instalații · Automatică și Informatică Aplicată

Proiect de Licență — Sesiunea Științifică Mai 2025

# FitAI

Sistem Inteligent IoT pentru  
Monitorizarea Activității Fizice  
și Recuperarea CNS

## Document de Specificare a Cerințelor Software

Software Requirements Specification (SRS)

|            |                                                                               |
|------------|-------------------------------------------------------------------------------|
| Autor      | Andrei Buruiană                                                               |
| Instituție | UTCB — Automatică și Informatică Aplicată                                     |
| Versiune   | 1.0 — Final                                                                   |
| Data       | 27 Aprilie 2025                                                               |
| Stare      | Aprobat                                                                       |
| Repository | <a href="https://github.com/AndreiBuruiana13">github.com/AndreiBuruiana13</a> |

## Istoricul Versiunilor

| Ver. | Data        | Modificări                                                                                                                               | Autor       |
|------|-------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 0.1  | 14 Oct 2024 | Creare schelet document; structură secțiuni conform IEEE 830. Completare secțiunilor 1.1-1.3; definit domeniul de aplicare.              | A. Buruiană |
| 0.2  | 07 Nov 2024 | Adăugate cerințele hardware (RF-010-RF-016) și constrângerile de interfață. Actualizat glosarul cu terminologia IoT și senzori.          | A. Buruiană |
| 0.3  | 02 Dec 2024 | Reformulat RF-001-RF-007 după feedback sesiune laborator. Adăugate cerințe de rate limiting și refresh token. Revizuit secțiunea 1.4.    | A. Buruiană |
| 0.4  | 18 Ian 2025 | Cerințe complete ML (RF-030-RF-035): specificat Random Forest, PAMAP2, format .pkl. Adăugat scor recuperare CNS cu formulă ponderată.    | A. Buruiană |
| 0.5  | 11 Feb 2025 | Adăugate cerințe frontend PWA (RF-040-RF-053). Completat secțiunea 5 — constrângeri tehnologice.                                         | A. Buruiană |
| 0.6  | 04 Mar 2025 | Revizuit secțiunea 4: adăugate metrici măsurabile de performanță (RNF-001-RNF-008) și cerințe de securitate detaliate.                   | A. Buruiană |
| 0.7  | 21 Mar 2025 | Adăugată matricea de trasabilitate (Anexa 6.1), format payload JSON (Anexa 6.2) și schema ERD simplificată (Anexa 6.3).                  | A. Buruiană |
| 0.8  | 03 Apr 2025 | Review intern: corecții terminologice, uniformizare coduri RF-XXX, completat câmpul <i>Metrică</i> pentru toate RNF-urile.               | A. Buruiană |
| 0.9  | 14 Apr 2025 | Revizuiți post-review: clarificat RF-026 (SSE vs. polling), actualizat RF-015 (buffer SPIFFS), adăugat criteriu acceptanță ML la RF-031. | A. Buruiană |
| 1.0  | 27 Apr 2025 | Versiune finală pentru depunere proiect de licență. Verificare completă consistență cerințe. Nicio modificare de conținut față de v0.9.  | A. Buruiană |

**Control versiuni:** Documentul este menținut în repository-ul Git al proiectului la calea docs/SRS.pdf. Fiecare versiune este tagată în Git cu convenția srs-vX.Y. Istoricul complet al modificărilor este disponibil prin `git log --follow docs/SRS.pdf`. Revizuirile minore (sub-versiuni) nu incrementează numărul de versiune principal.

# Cuprins

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Istoricul Versiunilor</b>                             | <b>i</b>  |
| <b>1 Introducere</b>                                     | <b>1</b>  |
| 1.1 Scop                                                 | 1         |
| 1.2 Domeniu de Aplicare                                  | 1         |
| 1.3 Definiții și Acronime                                | 1         |
| 1.4 Referințe                                            | 3         |
| 1.5 Structura Documentului                               | 3         |
| <b>2 Descriere Generală a Sistemului</b>                 | <b>4</b>  |
| 2.1 Perspectiva Produsului                               | 4         |
| 2.2 Funcțiile Principale ale Produsului                  | 4         |
| 2.3 Caracteristici și Clase de Utilizatori               | 5         |
| 2.4 Constrângeri Generale                                | 5         |
| 2.5 Ipoteze și Dependințe                                | 5         |
| <b>3 Cerințe Funcționale</b>                             | <b>7</b>  |
| 3.1 Modulul de Autentificare și Securitate               | 7         |
| 3.1.1 Înregistrare Utilizator                            | 7         |
| 3.1.2 Autentificare și Gestionarea Sesiunilor            | 7         |
| 3.2 Modulul Firmware ESP32 și Senzori                    | 8         |
| 3.3 Modulul Backend API și Bază de Date                  | 9         |
| 3.3.1 Gestionarea Sesiunilor de Antrenament              | 9         |
| 3.3.2 Ingestia Telemetriei                               | 9         |
| 3.4 Modulul ML — MotorFitAI                              | 10        |
| 3.5 Modulul Frontend — Dashboard PWA                     | 11        |
| <b>4 Cerințe Non-Funcționale</b>                         | <b>13</b> |
| 4.1 Performanță                                          | 13        |
| 4.2 Securitate                                           | 13        |
| 4.3 Fiabilitate și Disponibilitate                       | 14        |
| 4.4 Mentenabilitate                                      | 14        |
| 4.5 Portabilitate                                        | 15        |
| <b>5 Constrângeri de Proiectare și Interfețe Externe</b> | <b>16</b> |
| 5.1 Interfețe Hardware                                   | 16        |
| 5.2 Interfețe Software                                   | 16        |
| 5.3 Constrângeri Tehnologice                             | 16        |
| <b>6 Anexe</b>                                           | <b>18</b> |
| 6.1 Matricea de Trasabilitate Cerințe                    | 18        |
| 6.2 Format Payload JSON Telemetrie — Structură Comentată | 20        |
| 6.3 Schema Bazei de Date — ERD Simplificat               | 20        |
| 6.4 Criterii de Acceptanță Sintetice                     | 21        |

# 1 Introducere

---

## 1.1 Scop

Prezentul document constituie **Specificația Cerințelor Software (SRS)** pentru sistemul **FitAI** — o platformă inteligentă IoT dedicată monitorizării activității fizice, urmării parametrilor biologici în timp real și predicției nivelului de recuperare a sistemului nervos central (CNS).

Documentul are trei scopuri principale:

- **Definirea completă** și neambiguă a tuturor cerințelor funcționale și non-funcționale ale sistemului, astfel încât implementarea să poată fi realizată și verificată fără interpretări subiective;
- **Stabilirea unui contract de referință** între proiectant și evaluatori, asigurând că așteptările sunt aliniate înainte de începerea implementării efective;
- **Furnizarea bazei formale** pentru documentul de proiectare arhitecturală (SDS), pentru planul de testare și pentru backlog-ul de produs gestionat în Jira.

## 1.2 Domeniu de Aplicare

FitAI este un sistem distribuit compus din patru straturi distincte, fiecare cu responsabilități clare și interfețe bine definite:

1. **Stratul hardware (wearable)** — dispozitiv IoT bazat pe ESP32-C3 Supermini, echipat cu senzori biometrici (MAX30102 pentru HR și SpO<sub>2</sub>, MPU6050 pentru IMU) și modul GPS NEO-6M;
2. **Stratul backend** — server REST implementat în Python 3.11 cu framework FastAPI, persistența datelor în SQLite gestionată prin SQLAlchemy ORM;
3. **Stratul ML (MotorFitAI)** — motor de inteligență artificială pentru clasificarea automată a activității fizice (Random Forest pe PAMAP2) și calculul scorului de recuperare CNS;
4. **Stratul frontend (PWA)** — aplicație web progresivă cu vizualizări Chart.js, funcționalitate offline prin Service Worker, accesibilă de pe orice browser modern fără instalare.

Sistemul se adresează utilizatorilor sportivi și persoanelor active fizic care doresc date biometrice de precizie și recomandări personalizate de antrenament, fără dependență de servicii cloud proprietare sau abonamente recurente.

## 1.3 Definiții și Acronime

| Termen           | Definiție                                                                                                        |
|------------------|------------------------------------------------------------------------------------------------------------------|
| CNS              | Sistem Nervos Central — în FitAI: scor zilnic (0-100) care cuantifică nivelul de recuperare fiziologică          |
| HR               | <i>Heart Rate</i> — frecvența cardiacă, exprimată în bătăi pe minut (bpm)                                        |
| HRV              | <i>Heart Rate Variability</i> — variabilitatea intervalelor R-R; indicator al balanței sistemului nervos autonom |
| SpO <sub>2</sub> | Saturația periferică a oxigenului din sânge, exprimată procentual (valoare normală: $\geq 95\%$ )                |
| IoT              | <i>Internet of Things</i> — ecosistem de dispozitive fizice interconectate prin rețea                            |
| PWA              | <i>Progressive Web App</i> — aplicație web cu capabilități de instalare și funcționare offline                   |
| JWT              | <i>JSON Web Token</i> — standard RFC 7519 pentru transmiterea autentificată a informațiilor                      |
| RF               | <i>Random Forest</i> — ansamblu de arbori de decizie utilizat ca clasificator de activitate                      |
| PAMAP2           | Dataset public de activitate fizică cu senzori IMU și cardiac (UCI ML Repository, 52 subiecți)                   |
| IMU              | <i>Inertial Measurement Unit</i> — accelerometru 3-ax + giroscop 3-ax                                            |
| SSE              | <i>Server-Sent Events</i> — streaming unidirecțional HTTP server → client                                        |
| ORM              | <i>Object-Relational Mapping</i> — abstracție pentru baza de date (SQLAlchemy în FitAI)                          |
| MET              | <i>Metabolic Equivalent of Task</i> — unitate pentru estimarea cheltuielii calorice                              |
| ESP32-C3         | Microcontroller Espressif RISC-V 32-bit cu WiFi 2.4 GHz și BLE 5.0 integrat                                      |
| MAX30102         | Senzor optic de pulsoximetrie produs de Maxim Integrated; comunicare I <sup>2</sup> C                            |
| MPU6050          | IMU cu accelerometru 3-ax și giroscop 3-ax, produs de InvenSense; comunicare I <sup>2</sup> C                    |
| NEO-6M           | Modul GPS UART produs de u-blox; ieșire NMEA 0183; timp la fix: $\leq 60s$ (cold start)                          |

| Termen | Definiție                                                                                  |
|--------|--------------------------------------------------------------------------------------------|
| FCP    | <i>First Contentful Paint</i> — metrică de performanță web (primul element vizibil randat) |

## 1.4 Referințe

- IEEE Std 830-1998 — *Recommended Practice for Software Requirements Specifications*
- Murmann, P., Beckerle, M., Fischer-Hübner, S., Reinhardt, D. (2021). *Fitness tracking privacy. Pervasive and Mobile Computing*, Elsevier
- PAMAP2 Physical Activity Monitoring Dataset — UCI ML Repository (<https://archive.ics.uci.edu/dataset/231>)
- FastAPI Documentation v0.110 — <https://fastapi.tiangolo.com>
- Espressif ESP32-C3 Technical Reference Manual, Rev. 0.4
- scikit-learn 1.4 — RandomForestClassifier — <https://scikit-learn.org>
- RFC 7519 — JSON Web Token (JWT) — IETF, 2015

## 1.5 Structura Documentului

| Cap. | Titlu                   | Conținut                                                                              |
|------|-------------------------|---------------------------------------------------------------------------------------|
| 2    | Descriere Generală      | Contextul produsului, comparație cu soluții existente, clase de utilizatori, ipoteze  |
| 3    | Cerințe Funcționale     | 30+ cerințe RF-XXX organizate pe 5 module: Auth, Firmware, Backend, ML, Frontend      |
| 4    | Cerințe Non-Funcționale | Performanță (8 metrici), securitate, fiabilitate, mentenabilitate, portabilitate      |
| 5    | Constrângeri            | Interfețe hardware/software, constrângeri tehnologice cu justificări                  |
| 6    | Anexe                   | Matrice trasabilitate, format payload JSON comentat, schema ERD, glosar tehnic extins |

## 2 Descriere Generală a Sistemului

### 2.1 Perspectiva Produsului

FitAI este un sistem **standalone nou**, independent de orice platformă cloud terță proprietară. Spre deosebire de soluțiile comerciale existente, FitAI oferă transparență completă asupra algoritmilor, acces la datele brute și un cost hardware de ordinul a 15–20 EUR.

| Caracteristică        | FitAI                                 | Soluții comerciale        |
|-----------------------|---------------------------------------|---------------------------|
| Acces date brute      | Complet (SQLite local, export liber)  | Limitat sau absent        |
| Algoritmi ML          | Open-source, modificabili, auditabili | Proprietari, cutie neagră |
| Scor recuperare CNS   | Calculat local, formulă publicată     | Absent sau nedocumentat   |
| Cost hardware estimat | ~15–20 EUR (componente)               | 150–500 EUR (dispozitiv)  |
| Abonament obligatoriu | Nu                                    | Da (Whoop: ~\$30/lună)    |
| Cloud obligatoriu     | Nu — funcționează local complet       | Da                        |
| Open-source           | Da (MIT License)                      | Nu                        |

### 2.2 Funcțiile Principale ale Produsului

La nivel macro, FitAI îndeplinește șase funcții esențiale:

1. **Colectarea continuă** a datelor biometrice prin senzori hardware wearable (HR, SpO<sub>2</sub>, IMU 3-ax, GPS);
2. **Transmiterea în timp real** a telemetriei la serverul backend prin WiFi 2.4 GHz (REST POST, 1 Hz);
3. **Stocarea persistentă** a sesiunilor de antrenament și telemetriei în baza de date SQLite, cu schema versionată Alembic;
4. **Clasificarea automată** a activității fizice (6 clase) prin Random Forest antrenat pe dataset-ul PAMAP2;
5. **Calculul scorului de recuperare CNS** zilnic pe baza HRV, HR de repaus, calitate somn și sarcină de antrenament;

6. **Vizualizarea interactivă** a datelor printr-un dashboard web PWA cu grafice timp real, calendar heatmap și istoricul sesiunilor.

## 2.3 Caracteristici și Clase de Utilizatori

| Utilizator          | Profil                                                                               | Acces                        | Frecv.      |
|---------------------|--------------------------------------------------------------------------------------|------------------------------|-------------|
| Sportiv activ       | Alergare, ciclism, fitness zilnic; interesat de progres și recuperare pe termen lung | Toate modulele               | Zilnică     |
| Utilizator recreat. | Mișcare moderată; focus pe pași, calorii; 2-3 sesiuni/săptămână                      | Dashboard, History, Recovery | Săptămânală |
| Administrator       | Gestionare utilizatori, monitorizare server, analiză log-uri                         | Panel admin, logs, DB        | Ocazional   |

## 2.4 Constrângeri Generale

- Dispozitivul wearable trebuie să funcționeze pe baterie Li-Po 500 mAh cu **autonomie de minim 4 ore** de utilizare continuă (senzori activi, WiFi conectat, transmisie 1 Hz);
- Aplicația web trebuie să funcționeze **complet offline** pentru vizualizarea datelor istorice deja descărcate (Service Worker cache cu strategie Cache-First);
- **Datele utilizatorului nu sunt transmise la servicii cloud externe**; toată procesarea și stocarea se efectuează on-premise, pe serverul local al utilizatorului;
- Latența end-to-end de la senzor la afișare în dashboard nu trebuie să depășească **2 secunde** în condiții normale de rețea;
- Sistemul trebuie să suporte simultan **minim 10 utilizatori** activi fără degradare vizibilă de performanță (răspuns API <200 ms la P95).

## 2.5 Ipoteze și Dependințe

- Utilizatorul dispune de conexiune WiFi 2.4 GHz activă și stabilă în zona de antrenament pentru streaming live al telemetriei;
- Serverul backend rulează pe aceeași rețea locală sau este accesibil public (port forwarding sau deployment VPS) la un URL cunoscut de firmware;
- Browserul utilizatorului suportă Service Workers, IndexedDB și Web App Manifest — Chrome 90+, Firefox 88+, Safari 14+ (toate lansat din 2021);
- Temperatura ambientală se încadrează între 0°C și 45°C pentru funcționarea corectă a senzorilor optici MAX30102;



- Greutatea corporală a utilizatorului este introdusă manual în profil pentru estimarea caloriilor (formula Harris-Benedict cu factorul MET).

## 3 Cerințe Funcționale

Cerințele funcționale sunt identificate prin coduri **RF-XXX** unice și organizate pe module funcționale corespunzătoare epicelor din backlog-ul Jira al proiectului. Fiecare cerință specifică: ce trebuie să facă sistemul, *condiția de declanșare* și *rezultatul așteptat*.

**Legendă priorități:** **CRITICĂ** funcționalitate de bază, sistem inutilizabil fără ea  
**ÎNALTĂ** impact major asupra utilizabilității **MEDIE** valoare adăugată semnificativă  
**JOASĂ** nice-to-have pentru versiunea 1.0

### 3.1 Modulul de Autentificare și Securitate

#### 3.1.1 Înregistrare Utilizator

##### RF-001 — Creare cont utilizator

**Prioritate:** **CRITICĂ** **Modul:** Auth

**Descriere:** Sistemul trebuie să permită crearea unui cont nou prin furnizarea adresei de email, a unei parole și a unui nume de utilizator. Parola trebuie criptată cu bcrypt (cost factor  $\geq 12$ ) înainte de stocare. Nicio parolă în clar nu trebuie să apară în log-uri sau în răspunsurile API.

**Precondiție:** Emailul nu există deja în baza de date. **Postcondiție:** HTTP 201 + {user\_id, email, created\_at}.

##### RF-002 — Validare unicitate și complexitate

**Prioritate:** **CRITICĂ** **Modul:** Auth

**Descriere:** Sistemul trebuie să respingă înregistrarea dacă: (a) emailul este deja înregistrat → HTTP 409 cu mesaj generic fără a confirma existența contului; (b) parola are sub 8 caractere → HTTP 422; (c) emailul nu respectă formatul RFC 5322 → HTTP 422. Mesajele de eroare nu divulgă informații despre utilizatori existenți.

#### 3.1.2 Autentificare și Gestionarea Sesiunilor

##### RF-004 — Autentificare cu JWT

**Prioritate:** **CRITICĂ** **Modul:** Auth

**Descriere:** Sistemul trebuie să autentifice utilizatorul pe baza perechii email + parolă. La succes returnează: un **access token JWT** (expirat la 30 de minute, algoritm HS256) și un **refresh token opac** (expirat la 7 zile), stocat hash-uit în baza de date. Cheia secretă JWT este citită din variabila de mediu SECRET\_KEY.

**RF-005/006 — Refresh token și Logout****Prioritate:** CRITICĂ **Modul:** Auth

**RF-005:** Sistemul trebuie să permită reînnoirea access token-ului expirat prin prezentarea unui refresh token valid, fără re-autentificare. Refresh token-ul rămâne valid până la expirare sau logout.

**RF-006:** La logout, sistemul marchează refresh token-ul ca revocat în tabela refresh\_tokens. Orice tentativă ulterioară de refresh cu același token returnează HTTP 401.

**RF-007 — Rate limiting autentificare****Prioritate:** ÎNALTĂ **Modul:** Auth / Securitate

**Descriere:** Sistemul trebuie să blocheze temporar un IP după **5 tentative consecutive eșuate** de autentificare, pentru o perioadă de **15 minute**. Contorul se resetează la autentificare reușită. Răspuns la blocare: HTTP 429 cu header Retry-After: 900.

**3.2 Modulul Firmware ESP32 și Senzori****RF-010/011 — Citire MAX30102 și MPU6050****Prioritate:** CRITICĂ **Modul:** Firmware

**RF-010:** Firmware-ul trebuie să citească HR și SpO<sub>2</sub> de la MAX30102 prin I<sup>2</sup>C (adresă 0x57, 400 kHz) la **minim 1 Hz**. Algoritmul de calcul HR folosește detectarea vârfurilor pe semnalul IR filtrat trece-jos. Valorile invalide (HR  $\notin$  [30,250] bpm) sunt ignorate și logate.

**RF-011:** Firmware-ul trebuie să citească accelerometrul ( $\pm 8g$ , 3 axe) și giroscopul ( $\pm 500^\circ/s$ , 3 axe) de la MPU6050 prin I<sup>2</sup>C (adresă 0x68) la **minim 10 Hz**. Calibrarea offset-urilor se efectuează la pornire pe 2 secunde cu dispozitivul imobil.

**RF-012 — Citire NEO-6M GPS****Prioritate:** ÎNALTĂ **Modul:** Firmware

**Descriere:** Firmware-ul parsează sentințele NMEA (GGA + RMC) de la GPS prin UART (9600 baud, 8N1) și extrage: latitudine, longitudine, viteză (km/h), altitudine (m) și numărul de sateliți. Dacă fix GPS nu este disponibil în 30 de secunde de la pornire, câmpurile GPS sunt transmise ca null.

**RF-013/014 — Transmisie WiFi și Reconectare automată****Prioritate:** CRITICĂ **Modul:** Firmware

**RF-013:** Firmware-ul trimite datele colectate la POST /api/v1/telemetry prin HTTP, cu body JSON și header Authorization: Bearer {token}, la intervale de **1 secundă**. Timeout per cerere: 3 secunde.

**RF-014:** La pierderea conexiunii WiFi sau server, firmware-ul inițiază reconectarea cu **back-off exponențial** (1s, 2s, 4s, 8s, max 30s), fără a bloca citirea senzorilor.

#### RF-015 — Buffer local și LED status

**Prioritate:** **MEDIE** **Modul:** Firmware

**RF-015:** Firmware-ul stochează temporar în RAM/SPIFFS ultimele **60 pachete** în caz de pierdere a conexiunii. La reconectare, pachetele din buffer se retransmit în ordine cronologică.

**RF-016:** LED RGB indică starea: **verde continuu** = conectat și transmite; **galben pulsatoriu** = reconectare în curs; **roșu intermitent** = eroare senzor; **albastru** = boot/calibrare.

## 3.3 Modulul Backend API și Bază de Date

### 3.3.1 Gestionarea Sesiunilor de Antrenament

#### RF-020/021 — Start și Stop sesiune

**Prioritate:** **CRITICĂ** **Modul:** Backend / Sessions

**RF-020 POST /sessions/start:** Creează o înregistrare nouă în sessions cu status="active" și started\_at=now(). Returnează {session\_id, started\_at} cu HTTP 201. Dacă există deja o sesiune activă → HTTP 409.

**RF-021 POST /sessions/stop:** Calculează și stochează sumarizarea: durată totală, HR<sub>avg/min/max</sub>, calorii estimate (MET × greutate × durată), distanță GPS (Haversine cumulat). Setează status="completed" și ended\_at=now().

#### RF-022/023 — Listă și Detalii sesiune

**Prioritate:** **CRITICĂ** **Modul:** Backend / Sessions

**RF-022 GET /sessions:** Returnează lista paginată (parametri page, limit, activity\_type) a sesiunilor utilizatorului curent, sortate descrescător după started\_at.

**RF-023 GET /sessions/{id}:** Returnează sumarizarea completă, seria temporală HR ([{timestamp, hr\_bpm}]), traseul GPS și distribuția ML. Acces la sesiunile altor utilizatori → HTTP 403.

### 3.3.2 Ingestia Telemetriei

#### RF-024/025/026 — Telemetrie: ingestie, validare și streaming

**Prioritate:** **CRITICĂ** **Modul:** Backend / Telemetry

**RF-024:** Acceptă și stochează pachete JSON de la ESP32 în tabela telemetry. Câmpuri obligatorii: session\_id, timestamp, hr\_bpm, spo2\_pct, accel (x,y,z), gyro (x,y,z). Câmpul gps este opțional. Răspuns: HTTP 201.

**RF-025:** Respinge pachete cu valori în afara intervalelor admise: HR  $\notin$  [30,250], SpO<sub>2</sub>  $\notin$  [70,100], coordonate GPS invalide → HTTP 422.

**RF-026:** Expune endpoint de polling la 1 Hz care returnează cel mai recent pachet al sesiunii active. Dacă nu există sesiune activă → HTTP 204.

### 3.4 Modulul ML — MotorFitAI

#### RF-030/031 — Clasificare activitate și nivel de încredere

**Prioritate:** **CRITICĂ** **Modul:** ML

**RF-030:** Clasifică activitatea fizică în una din clasele: MERS, ALERGAT, CICLISM, REPAUS, URCAT\_SCĂRI, STAT\_JOS, folosind un **Random Forest** cu 100 arbori antrenat pe PAMAP2. Caracteristici: statistici fereastră glisantă 2s pe axele IMU + HR (medie, deviație standard, min, max, percentile).

**RF-031:** Returnează probabilitatea clasificării (confidence, 0.0–1.0). Dacă confidence < 0.55 → clasa NECUNOSCUT cu avertisment vizual. Criterii de acceptanță ML: acuratețe  $\geq 85\%$  pe setul de validare PAMAP2.

#### RF-032/033 — Scor recuperare CNS și Recomandare intensitate

**Prioritate:** **CRITICĂ** **Modul:** ML / Recovery

**RF-032:** Calculează zilnic scorul CNS  $\in$  [0,100] prin formula ponderată:

$$\text{CNS} = 0.35 \cdot f(\text{HRV}) + 0.25 \cdot f(\text{HR}_{\text{rest}}) + 0.25 \cdot f(\text{sleep}) + 0.15 \cdot f(\text{load})$$

unde  $f(\cdot)$  este o funcție de normalizare la intervalul [0,1]. Scor  $\geq 75$ : recuperat; 50–74: parțial; <50: sub-recuperat.

**RF-033:** Generează recomandare: **REPAUS** (CNS<35), **UȘOR** (35–55), **MODERAT** (55–75), **INTENS** (>75), cu tip activitate sugerat și durată estimată.

#### RF-034/035 — Tendință HR și Persistare model

**Prioritate:** **MEDIE** **Modul:** ML

**RF-034:** Calculează regresia liniară a HR de repaus pe 30 de zile: **îmbunătățire** (pantă < -0.2 bpm/zi), **stagnare** ( $\in$  [-0.2, +0.2]), **deteriorare** (> +0.2 bpm/zi).

**RF-035:** Modelul RF se salvează ca model\_rf\_v1.pkl și se încarcă la startup. Re-antrenarea este offline. La startup se loghează: model\_version, train\_date, accuracy\_val.

### 3.5 Modulul Frontend — Dashboard PWA

#### RF-040/041/042/043 — Dashboard principal

**Prioritate:** **CRITICĂ** **Modul:** Frontend

**RF-040:** Afișează live (refresh 1 Hz): HR curent (bpm), SpO<sub>2</sub> (%), pași estimați, calorii cumulate. Culori semantice: HR>160 → roșu, 120–160 → portocaliu, <120 → verde.

**RF-041:** Bar chart Chart.js cu distribuția timpului în 5 zone HR (Z1–Z5), actualizat la final de sesiune.

**RF-042:** Pie/doughnut chart cu distribuția procentuală a activităților detectate ML în ziua curentă.

**RF-043:** Lista ultimelor 5 sesiuni cu: dată, durată, activitate principală, HR<sub>avg</sub>, calorii.

#### RF-044/045/046 — Istoricul sesiunilor și Modal detalii

**Prioritate:** **CRITICĂ** **Modul:** Frontend

**RF-044:** Pagina History afișează toate sesiunile paginate (10/pagină), filtrabile după perioadă și tip activitate.

**RF-045:** La click pe sesiune se deschide modal cu: grafic HR line chart (Chart.js), statistici HR min/max/avg, durată, distanță GPS, calorii, distribuție ML (stacked bar).

**RF-046:** Graficul HR din modal colorează segmentele pe zone cardiacă (Z1–Z5) pentru citire vizuală rapidă.

#### RF-047/048/049/050 — Recuperare CNS

**Prioritate:** **CRITICĂ** **Modul:** Frontend

**RF-047:** Pagina Recovery afișează scorul CNS zilnic (0–100) cu indicator culoare și etichetă verbală.

**RF-048:** Calendar heatmap cu scorul CNS pentru ultimele 30 de zile; hover afișează valoarea exactă și recomandarea.

**RF-049:** Afișează HRV matinal, HR de repaus, calitate somn cu tendința față de media 7 zile.

**RF-050:** Recomandare de intensitate generată de MotorFitAI, cu tip activitate și durată sugerată.

#### RF-051/052 — PWA: Service Worker și Manifest

**Prioritate:** **ÎNALTĂ** **Modul:** Frontend / PWA

**RF-051:** Service Worker (strategie Cache-First pentru statice, Network-First pentru API) cacheaza resursele statice și ultimele 30 zile de date. Banner offline când nu există conexiune.

**RF-052:** Fișier manifest.webmanifest complet (name, short\_name, icons 192 + 512 px, theme\_color #028090, display: standalone) permite instalarea pe Android, iOS și desktop Chrome.

## 4 Cerințe Non-Funcționale

### 4.1 Performanță

| ID      | Cerință                                                     | Metrică    | Prior.  |
|---------|-------------------------------------------------------------|------------|---------|
| RNF-001 | Latența de ingestie telemetrie la backend (per pachet, P95) | <200 ms    | ÎNALTĂ  |
| RNF-002 | Latența end-to-end senzor → afișare dashboard               | <2 secunde | ÎNALTĂ  |
| RNF-003 | Timp de clasificare activitate ML per fereastră (CPU only)  | <50 ms     | ÎNALTĂ  |
| RNF-004 | Timp de calcul scor CNS zilnic complet                      | <500 ms    | MEDIE   |
| RNF-005 | First Contentful Paint (FCP) la încărcarea inițială         | <3 secunde | MEDIE   |
| RNF-006 | Utilizatori activi simultan fără degradare                  | ≥10        | MEDIE   |
| RNF-007 | Autonomia bateriei wearable la utilizare continuă           | ≥4 ore     | CRITICĂ |
| RNF-008 | Rata de eșantionare IMU (MPU6050)                           | ≥10 Hz     | CRITICĂ |

### 4.2 Securitate

Cerințele de securitate sunt tratate ca o preocupare transversală, nu izolată. Principalele cerințe sunt:

- **Autentificare obligatorie:** Toate endpoint-urile (cu excepția /auth/login și /auth/register) cer un JWT valid în header-ul Authorization: Bearer. Cererile fără token valid primesc HTTP 401.
- **Stocare parole:** Parolele sunt stocate exclusiv ca hash bcrypt (cost factor ≥12). Nicio parolă în clar nu trebuie să apară în baza de date, fișierele de log sau răspunsurile API.
- **Rate limiting global:** Toate endpoint-urile publice sunt limitate la 100 req/min/IP. Endpoint-ul de login are limită suplimentară de 5 tentative eșuate/15 min/IP.
- **Izolare date utilizatori:** Fiecare cerere care accesează resurse (sesiuni, telemetrie, recuperare) validează că user\_id din JWT coincide cu proprietarul resursei. Orice



neconcordanță returnează HTTP 403.

- **Revocarea token-urilor:** Refresh token-urile sunt stocate hash-uite în tabela `refresh_tokens` și marcate ca revocate la logout. Token-ul de access (JWT stateless) expiră în 30 de minute.
- **Configurare prin variabile de mediu:** Cheia secretă JWT, calea bazei de date, portul serverului și parametrii de email sunt configurate exclusiv prin fișier `.env`; nu sunt hardcodate în codul sursă.
- **HTTPS în producție:** Deployment-ul pe VPS va folosi HTTPS cu certificat Let's Encrypt. Comunicarea firmware ↔ server pe rețea locală poate folosi HTTP în versiunea 1.0.

### 4.3 Fiabilitate și Disponibilitate

- Firmware-ul ESP32 gestionează reconectarea automată la WiFi și server cu back-off exponențial, fără intervenție manuală și fără oprirea citirii senzorilor;
- Pierderea unui pachet de telemetrie nu corupă și nu termină sesiunea curentă; sesiunea rămâne activă până la apelul explicit `/sessions/stop`;
- Backend-ul FastAPI este configurat cu supervisor sau systemd pentru restart automat la crash;
- Datele sesiunilor active sunt persistate în SQLite la fiecare pachet primit, nu la finalul sesiunii — un restart al serverului în mijlocul sesiunii nu pierde datele deja ingerate;
- Uptime țintă pentru serverul backend în faza de demonstrație:  $\geq 99\%$  în fereastra orară 08:00–22:00.

### 4.4 Mentenabilitate

- Codul backend este organizat modular: `auth/`, `sessions/`, `telemetry/`, `ml/`, `recovery/`, `core/`; fiecare modul conține `router.py`, `models.py`, `schemas.py`;
- Firmware-ul PlatformIO are fișiere separate per responsabilitate: `sensors.cpp` (MAX30102, MPU6050), `gps.cpp` (NEO-6M, NMEA), `wifi_client.cpp` (transmisie REST);
- Toate endpoint-urile API sunt documentate automat prin Swagger UI (FastAPI built-in), accesibil la `/docs`; versiunea Redoc la `/redoc`;
- Schema bazei de date este gestionată prin migrații **Alembic**, versionabile în Git, aplicabile incremental;
- Fiecare modul backend are teste unitare cu acoperire de minim **60%** măsurată prin `pytest-cov`.

## 4.5 Portabilitate

- Serverul backend rulează pe Linux Ubuntu 22.04+ și poate fi containerizat cu Docker (Dockerfile inclus în repository);
- Frontend-ul PWA funcționează corect în **Chrome 90+, Firefox 88+ și Safari 14+** — ultimele 2 versiuni majore ale fiecărui browser;
- Firmware-ul este compilat cu PlatformIO CLI, independent de IDE, reproductibil pe orice mașină cu `pio run`;
- Baza de date SQLite poate fi migrată la PostgreSQL prin modificarea `DATABASE_URL` și rularea migrațiilor Alembic — fără modificări de cod aplicație.

## 5 Constrângeri de Proiectare și Interfețe Externe

### 5.1 Interfețe Hardware

| Componentă    | Protocol         | Parametri cheie       | Observații                                                                             |
|---------------|------------------|-----------------------|----------------------------------------------------------------------------------------|
| MAX30102      | I <sup>2</sup> C | Adresă 0x57, 400 kHz  | Alimentare 1.8 V I/O; LED 5 V; semnal IR pentru HR; semnal Red pentru SpO <sub>2</sub> |
| MPU6050       | I <sup>2</sup> C | Adresă 0x68 (AD0=GND) | ±8g accel; ±500°/s gyro; DLPF setat la 10 Hz; DMP dezactivat                           |
| NEO-6M        | UART             | 9600 baud, 8N1        | NMEA 0183 GGA+RMC; first fix: ≤60s cold start, ≤5s warm start                          |
| Li-Po 500 mAh | —                | 3.7 V nominal         | Circuit protecție obligatoriu; conector JST PH 2.0; încărcare TP4056                   |
| LED RGB       | GPIO             | 3 pini PWM            | Rezistoare serie 100Ω; verde/galben/roșu/albastru conform RF-016                       |

### 5.2 Interfețe Software

- Backend-ul expune exclusiv **REST API JSON** la portul 8000 (configurabil prin .env); prefixul URL al tuturor endpoint-urilor este /api/v1/;
- Frontend-ul comunică cu backend-ul exclusiv prin HTTP(S) cu JWT în header-ul Authorization; token-urile nu sunt stocate în localStorage (vulnerabil XSS), ci în memorie JS sau sessionStorage;
- Modelul ML este serializat în format scikit-learn Pickle (model\_rf\_v1.pkl) și versioned în Git LFS; se include și fișierul de metadate model\_metadata.json (acuratețe, date antrenare, versiune features);
- Schema bazei de date SQLite este gestionată prin migrații Alembic; calea fișierului DB este configurată prin variabila de mediu DATABASE\_URL (format: sqlite:///./data/fitai.db).

### 5.3 Constrângeri Tehnologice

Toate deciziile tehnologice au fost luate cu justificare explicită — nu prin convenție sau familiaritate, ci pe baza cerințelor concrete ale sistemului.

| Componentă               | Tehnologie aleasă și justificare                                                                                                                      |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microcontroller wearable | <b>ESP32-C3 Supermini</b> — raport cost/performanță (~2 EUR), WiFi 2.4 GHz integrat, ecosistem PlatformIO matur, consum redus în mod modem sleep      |
| Framework firmware       | <b>PlatformIO + Arduino</b> — portabilitate, biblioteci disponibile pentru MAX30102/MPU6050/NEO-6M, suport FreeRTOS pentru multitasking               |
| Backend web framework    | <b>Python FastAPI</b> — performanță async nativă (ASGI/uvicorn), validare automată Pydantic, generare Swagger fără configurație, tipuri Python native |
| Bază de date             | <b>SQLite + SQLAlchemy ORM</b> — deployment single-file fără server DB, adecvat ≤10 utilizatori, migrare facilă la PostgreSQL prin schimbare URL      |
| ML framework             | <b>scikit-learn RandomForestClassifier</b> — interpretabilitate ridicată, performanță dovedită pe PAMAP2, inferență <50 ms pe CPU fără GPU            |
| Vizualizare date         | <b>Chart.js 4.x</b> — integrare nativă în vanilla JS fără framework SPA, bundle <200 KB, API declarativ simplu                                        |
| Autentificare            | <b>JWT (python-jose)</b> — stateless, funcționează cu PWA offline, revocabilitate prin refresh token stocat server-side                               |

**Rațiunea alegerii stivei tehnologice:** Toate deciziile urmăresc principiul *complexitate minimă suficientă*. FitAI este un prototip academic, nu un produs la scară. SQLite în locul PostgreSQL, vanilla JS în locul React, scikit-learn în locul PyTorch — fiecare alegere reduce suprafața de configurare și dependențele externe, permițând unui singur dezvoltator să livreze un sistem complet funcțional în intervalul unui proiect de licență. Stiva este modulară: fiecare componentă poate fi înlocuită independent fără a afecta celelalte straturi.

## 6 Anexe

---

### 6.1 Matricea de Trasabilitate Cerințe

Tabelul următor asociază fiecare cerință funcțională cu modulul de implementare, fișierul sursă principal și testul de verificare corespunzător. Trasabilitatea completă permite identificarea rapidă a impactului oricărei modificări.

| ID         | Descriere scurtă       | Fișier implementare  | Test asociat                  |
|------------|------------------------|----------------------|-------------------------------|
| RF-001/002 | Creare cont + validare | auth/router.py       | test_auth::test_register      |
| RF-004/005 | Login + refresh JWT    | auth/router.py       | test_auth::test_login         |
| RF-006/007 | Logout + rate limiting | auth/middleware.py   | test_auth::test_logout        |
| RF-010/011 | MAX30102 + MPU6050     | firmware/sensors.c   | Test hardware manual          |
| RF-012/013 | GPS + transmisie WiFi  | firmware/wifi_client | Integration test ESP32↔server |
| RF-015/016 | Buffer + LED status    | firmware/buffer.c    | Test manual + osciloscop      |
| RF-020/021 | Start/stop sesiune     | sessions/router.py   | test_sessions                 |
| RF-024/025 | Ingestie + validare    | telemetry/router.py  | test_telemetry                |
| RF-030/031 | Clasificare RF + conf. | ml/classifier.py     | test_ml::test_classify        |
| RF-032/033 | Scor CNS + recom.      | recovery/calculator  | test_recovery                 |
| RF-034/035 | Tendință HR + model    | ml/analytics.py      | test_ml::test_trend           |
| RF-040-043 | Dashboard live         | frontend/dashboard   | E2E Playwright (manual)       |
| RF-044-046 | History + modal        | frontend/history.js  | E2E Playwright (manual)       |
| RF-047-050 | Recovery CNS + heatmap | frontend/recovery.   | E2E Playwright (manual)       |
| RF-051/052 | Service Worker + PWA   | frontend/sw.js       | Lighthouse PWA audit          |

## 6.2 Format Payload JSON Telemetrie — Structură Comentată

Structura completă a pachetului JSON transmis de firmware la POST /api/v1/telemetry la fiecare secundă de sesiune activă. Câmpurile marcate cu // optional sunt omise dacă valoarea nu este disponibilă.

```
{
  "session_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", // UUID sesiune active
  "timestamp": "2025-04-27T07:30:15.123Z", // ISO 8601 UTC
  "hr_bpm": 148, // integer, interval valid: [30, 250]
  "spo2_pct": 97.5, // float, interval valid: [70.0, 100.0]
  "accel": {
    "x": 0.12, "y": 9.81, "z": 0.05 // m/s², range ±8g
  },
  "gyro": {
    "x": 0.01, "y": -0.02, "z": 0.00 // °/s, range ±500°/s
  },
  "gps": { // optional – null dacă fix GPS indisponibil
    "lat": 44.4268, "lon": 26.1025,
    "speed_kmh": 9.2, "alt_m": 85.0
  }
}
```

**Validare server:** Backend-ul respinge (HTTP 422) pachetele cu câmpuri obligatorii lipsă sau valori în afara intervalelor. Câmpul gps poate fi complet omis sau setat null. Timestamp-ul trebuie să fie în UTC; o diferență de sincronizare >60s față de serverul backend generează un avertisment în log.

## 6.3 Schema Bazei de Date — ERD Simplificat

Tabelul descrie structura relațională a bazei de date SQLite. Toate tabelele au câmpul id de tip TEXT (UUID v4), generat server-side. Indexuri critice pentru performanță: sessions(user\_id), telemetry(session\_id, timestamp), daily\_recovery(user\_id, date). Migrațiile de schemă sunt gestionate prin **Alembic** și versionabile în Git.

| Tabelă         | Câmpuri principale                                                                                         | Relații și note                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| users          | id, email (UNIQUE), password_hash, name, weight_kg, created_at                                             | Entitate rădăcină. 1 → N sessions, 1 → N daily_recovery                                    |
| sessions       | id, user_id (FK), status (active/completed), started_at, ended_at, summary_json                            | N → 1 users; 1 → N telemetry. Index pe (user_id, started_at DESC)                          |
| telemetry      | id, session_id (FK), timestamp, hr_bpm, spo2_pct, accel_json, gyro_json, gps_json, activity_ml, confidence | N → 1 sessions. Tabelă cu volum mare — ~3600 rânduri/oră. Index pe (session_id, timestamp) |
| daily_recovery | id, user_id (FK), date (UNIQUE/user), cns_score, hrv_ms, hr_rest_bpm, sleep_quality, recommendation        | N → 1 users. Un rând per utilizator per zi; recalculat dacă există date noi                |
| refresh_tokens | id, user_id (FK), token_hash (UNIQUE), issued_at, expires_at, revoked_at (NULL = activ)                    | N → 1 users. Tokens revoked la logout; expirate șterse periodic prin job cron              |

## 6.4 Criterii de Acceptanță Sintetice

Înainte de livrarea versiunii 1.0, fiecare modul trebuie să treacă următoarele criterii minime de acceptanță:



| Modul         | Criteriu de acceptanță                                                            | Metodă verificare        |
|---------------|-----------------------------------------------------------------------------------|--------------------------|
| Auth          | Autentificare reușită în <500 ms; logout invalidează token în <100 ms             | pytest + manual          |
| Firmware      | Datele HR recepționate pe server la 1 Hz $\pm$ 100 ms pe durata a 30 min continuă | Test hardware            |
| Backend API   | Toate endpoint-urile returnează răspuns în <200 ms la P95 cu 10 clienți simultan  | Locust load test         |
| ML Classifier | Acuratețe $\geq$ 85% pe setul de test PAMAP2 (hold-out 20%)                       | pytest + sklearn metrics |
| Recovery CNS  | Scorul CNS calculat în <500 ms; formula reprodusă conform specificației           | pytest unit              |
| Frontend PWA  | Scor Lighthouse PWA $\geq$ 90; FCP <3s pe conexiune 4G simulată                   | Lighthouse CI            |

### Document Finalizat — FitAI SRS v1.0

Acest document reprezintă specificația completă a cerințelor pentru sistemul FitAI v1.0, elaborată conform standardului IEEE 830-1998.

Elaborat de

Sesiunea Științifică

**Andrei Buruiană**

**Mai 2025**

UTCB — AIA

UTCB București

[github.com/AndreiBuruiana13](https://github.com/AndreiBuruiana13) · [docs/SRS.pdf](#) · tag: srs-v1.0